Last lecture, we went over decoders and the algorithm behind how they work. We will look at the final type of combinational circuit that behaves similarly to how a decoder does.

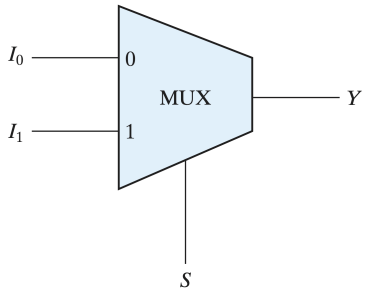
Multiplexer (MUX): one of the most useful building blocks in all of digital design. Out of several incoming data lines, pick one and send it to a single output. Which line gets picked is controlled by a separate set of **selection lines**. Because a multiplexer steers one chosen input to the output, it's also called a **data selector**

MUX Properties:

Selection lines n	Data inputs 2^n	Name
1	2	2-to-1 MUX
2	4	4-to-1 MUX
3	8	8-to-1 MUX
4	16	16-to-1 MUX

A multiplexer normally has 2^n data input lines and n selection lines. The reason is the same counting fact we used for decoders. With n selection bits you can make 2^n different patterns, and each pattern points at exactly one of the 2^n inputs. In every case there is exactly one output line. The whole point is to funnel many inputs down to one.

The 2-1 MUX:



The 2-1 MUX is the smallest MUX. It has two data inputs, one selection line, one output. We'll call the inputs I_0 and I_1, the selection line S, and the output Y

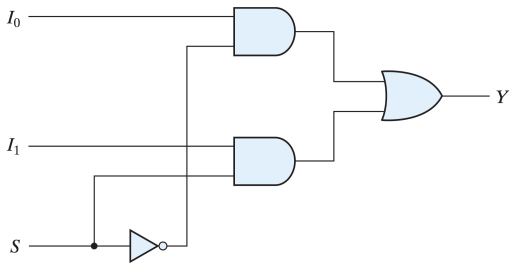
- **When S = 0, the output Y equals I_0**
- **When S = 1, the output Y equals I_1**

This kind of table on the right is called a function table. It is not a standard truth table.

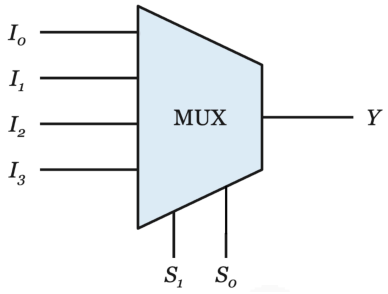
S	Y
0	I_0
1	I_1

How do we build this with logic gates? We need a way to "open a gate" for the chosen input and "close the gate" for the other.

- Feed I_0 into an AND gate along with S'. This gate is open only when S = 0.
- Feed I_1 into a second AND gate along with S. This gate is open only when S = 1.
- Combine the two AND gate outputs with an OR gate to form Y.



The 4-1 MUX:

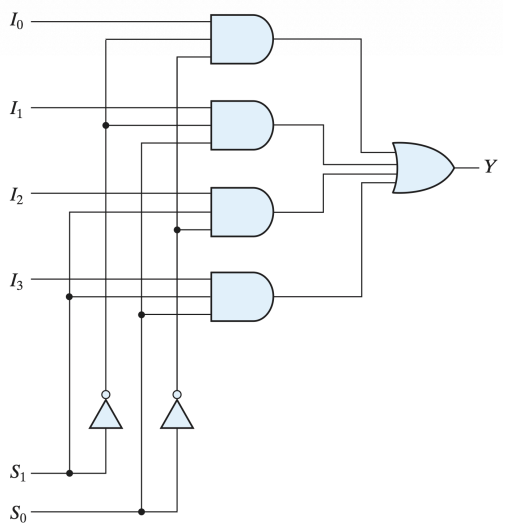


A 4-to-1 multiplexer has four data inputs, two selection lines and one output Y. If the selection lines spell out the binary number k, then input I_K is the one connected to the output. So, selection = 01 picks I_1 because 01 in binary is equivalent to 1 in decimal.

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

The construction follows the same recipe as the 2-to-1, just scaled up:

- There are four AND gates, one per data input.
- Each AND gate has three inputs: its data input, plus the right combination of S_1, S_0 (true or complemented) that matches its number. This combination is exactly what a decoder produces.
- All four AND gate outputs feed one big OR gate, which produces Y.



Worked Example 1 — Following a selection through a 4-to-1 MUX

Problem. A 4-to-1 MUX has data inputs $I_0 = 1$, $I_1 = 0$, $I_2 = 1$, $I_3 = 1$. The selection lines are set to $S_1S_0 = 11$. What is the output Y?

Step 1 — Read the selection as a number. $S_1S_0 = 11$ is binary for 3. So the multiplexer selects input I_3.

Step 2 — Look up that input's value. We're told $I_3 = 1$.

Step 3 — Output. $Y = I_3 = 1$.

Notice the values of I_0, I_1, I_2 are completely irrelevant here — they're on the closed gates. Only the selected input matters.

A multiplexer can implement any Boolean function directly, often with a single chip and no extra gates. A 2^n-to-1 multiplexer with its selection lines driven by the input variables will, for each

combination of those variables, connect one specific data input to the output. If we set each data input to the value the function should have for that combination, the multiplexer reproduces the function.

To build a function of n variables:

- take a 2^n-to-1 MUX, connect the n variables to the selection lines, and set data input I_K to the function's output value for row k of the truth table.

Worked Example 3 — A 3-variable function with an 8-to-1 MUX

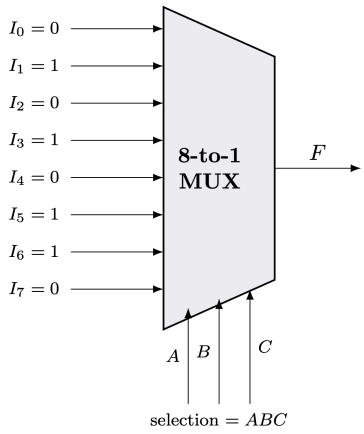
Problem. Implement $F(A, B, C) = \sum(1, 3, 5, 6)$ using an 8-to-1 multiplexer.
Step 1 — Choose the MUX. Three variables means $2^3 = 8$ rows, so we use an 8-to-1 MUX. Connect A, B, C to the selection lines $S_2S_1S_0$ (with A as the most significant).
Step 2 — Write out the truth table. $F = 1$ for minterms $1, 3, 5, 6$ and 0 elsewhere:

Row	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

Step 3 — Wire the data inputs. Set each I_k equal to F in row k:

$$I_0 = 0,\ I_1 = 1,\ I_2 = 0,\ I_3 = 1,\ I_4 = 0,\ I_5 = 1,\ I_6 = 1,\ I_7 = 0.$$

That's the whole circuit — one 8-to-1 MUX with each data pin tied to a constant 0 or 1.



To implement a function of n variables, you can often use a smaller MUX with only n−1 selection lines. The idea: use n−1 of the variables for selection, and let each data input be a simple function (0, 1, the last variable, or its complement) of the one leftover variable.

- Pick one variable to be the "leftover" (usually the least significant, here C). The other n−1 variables drive the selection lines.
- Write the truth table, grouping rows into pairs that share the same selection value. Each pair differs only in the leftover variable.
- For each pair, compare the two function values and figure out what the data input should be in terms of the leftover variable.

Worked Example 4 — Same function, smaller MUX

Problem. Implement the same $F(A, B, C) = \sum(1, 3, 5, 6)$, but now with a *4-to-1* MUX. Use A and B as selection lines and let C be the leftover variable.

Step 1 — Arrange the truth table so C varies within each pair. We group the eight rows into four pairs by their AB value:

A	B	C	F	MUX input	What to wire it to
0	0	0	0	I_0	F is 0 then 1 $\Rightarrow I_0 = C$
0	0	1	1		
0	1	0	0	I_1	F is 0 then 1 $\Rightarrow I_1 = C$
0	1	1	1		
1	0	0	0	I_2	F is 0 then 1 $\Rightarrow I_2 = C$
1	0	1	1		
1	1	0	1	I_3	F is 1 then 0 $\Rightarrow I_3 = \overline{C}$
1	1	1	0		

Step 2 — Read off the data inputs.

$$I_0 = C, \quad I_1 = C, \quad I_2 = C, \quad I_3 = \overline{C}.$$

Step 3 — Wire it up. Connect A and B to the selection lines, tie I_0, I_1, I_2 to the C line, and tie I_3 to $\overline{C}$ (one inverter needed). This implements the exact same function with a MUX half the size of the one in Example 3.

Check. Try $ABC = 110$ (row 6, where F should be 1). Selection $AB = 11$ picks $I_3 = \overline{C} = \overline{0} = 1$. Correct! Now try $ABC = 111$ (row 7, F should be 0): selection 11 picks $I_3 = \overline{C} = \overline{1} = 0$. Correct again.